

**Lab Goal :** This lab was designed to teach you how to use recursion to solve a connection problem.

**Lab Description :** Take a group of provided connections and create a graph of all connections. Search this graph with two provided nodes to see if paths exist in the graph between the two provided nodes. Print all paths between the 2 provided nodes. You will use HashMap and HashSet for most of the code.

**List of connections : AB BC CD DE CE**  
**Is A connected to E? Yes – paths A B C D E and A B C E**

### Sample Data :

```
9
CA XY RS YS ST TB AB BD RJ DC
CD
PQ QX AX BH CH DX EX FX GH AB BC CD DE AE CE FD TH
PT
AE EI IO OU BC CD DF FG
AG
HI HJ HK KQ KM KN MO MP MQ
HQ
AB CD EF GH CB ED GF HI
AI
TV XY AZ XT JK KL LT JX MN TN JL NO OP PT NX VZ
VZ
AB BC CD DE EF FG GH HI IJ AE AC JZ FZ AZ
AZ
NO PQ RS TU OU RP AB CD EF GH AH CE NS FA GQ DT
DT
IX VX CX DX MX LX BY
IB
```

### Files Needed ::

```
AllPathsGraph.java
AllPathsGraphRunner.java
graph3.dat
```

### Sample Output :

```
C to D == [C D, C A A B B T D, C A D]
P to T == [ P Q Q X X A A B B C C D E H H T]
A to G == no
H to Q == [ H I J K K Q, H I J K K Q M M P Q]
A to I == [ A B B C C D D E E F F G G H H I]
V to Z == [V Z, V T Z]
A to Z == [A Z, A B B C C D D E E F F G G H H I I J J Z, A B B C C D D E E F F G Z, A
B Z]
D to T == [D T, D C C E E F F A A B H H G G Q Q P P R R S S N N O O U U T, D C T]
I to B == no
```